

DEPARTMENT OF CSE

COURSE CODE : CSE 402

COURSE TITLE : Operating System Lab

REPORT : 05

REPORT NAME : Priority Scheduling

SUBMITTED BY:

NAME : SUJANA MAHMUDA NITE

STUDENT ID. : 23101120

SEMESTER : 4th YEAR 1st SEMESTER

SECTION : C-1

SUBMITTED TO:

ATIA RAHMAN ORTHI

UAP

DATE OF SUBMISSION: 12-10-2026

Problem Statement

Implement and study Priority CPU Scheduling using the given processes, Arrival Time (AT), Burst Time (BT), and Priority values. Observe the effect of different time quantum values, calculate Completion Time (CT), Turnaround Time (TAT), Waiting Time (WT), Average TAT and Average WT, and represent the scheduling result using a Gantt Chart where applicable.

Objective

The objective of this lab is to understand how priority-based CPU scheduling selects processes and how process priority affects execution order and performance. The experiment also examines the effect of Time Quantum (TQ) values in the provided priority scheduling program and calculates the standard scheduling metrics for each process.

Working Principle of Priority Scheduling

Priority Scheduling is a CPU scheduling technique in which processes are executed based on their assigned priority levels. In the first program, the scheduler chooses a non-empty priority queue with the lowest priority number, indicating that a smaller PR value means a higher priority. Processes having equal priority are maintained in a queue. If a process cannot finish within its allocated time quantum, it is placed back into the corresponding queue to continue execution later.

The second program demonstrates another priority-scheduling example using predefined Completion Time values and a Gantt Chart. From these values, the program determines the Turnaround Time (TAT), Waiting Time (WT), Average Turnaround Time, and Average Waiting Time.

Advantages of Priority Scheduling

- High-priority processes receive CPU service before lower-priority processes.
- The scheduling behavior can be controlled by assigning different priority levels to processes.
- It is suitable for systems where tasks have different levels of importance.
- Using a time quantum allows processes with equal priority to share CPU time fairly.

Disadvantages of Priority Scheduling

- Low-priority processes may experience excessive delays or starvation.
- Assigning and managing appropriate priority levels can be challenging.
- A very short time quantum may cause frequent context switching and reduce CPU efficiency.
- Priority inversion may occur when a high-priority process is delayed because a lower-priority process holds a required resource.

Solution to Starvation: Starvation can be minimized through a technique called *aging*. In this method, the priority of a waiting process is gradually increased over time, allowing it to receive CPU service eventually.

Given Data

Priority Scheduling with TQ = 1 and TQ = 3

Process	AT	BT	Priority
P1	3	4	2
P2	0	4	0
P3	2	3	1
P4	1	6	1
P5	4	2	3

Source Code - Case 1

```
from collections import deque

process = ["P1","P2","P3","P4","P5"]

at = [3,0,2,1,4]

bt = [4,4,3,6,2]

pr = [2,0,1,1,3]


def table(title, ct):

    tat = [ct[i]-at[i] for i in range(5)]
```

```
    wt = [tat[i]-bt[i] for i in range(5)]

    print("\n", title)
    print("P AT BT PR CT TAT WT")

    for i in range(5):
        print(process[i], at[i], bt[i], pr[i], ct[i], tat[i], wt[i])

    print("Avg TAT =", sum(tat)/5)
    print("Avg WT =", sum(wt)/5)

def priority(tq):
    rem = bt.copy()
    ct = [0]*5
    q = {0: deque(), 1: deque(), 2: deque(), 3: deque()}
    added = set()
    time = done = 0

    while done < 5:

        for i in range(5):
            if i not in added and at[i] <= time:
                q[pr[i]].append(i)
                added.add(i)

        if not any(q.values()):
            time += 1
            continue
```

```
        p = min(k for k in q if q[k])
        i = q[p].popleft()

        run = min(tq, rem[i])

        for _ in range(run):
            rem[i] -= 1
            time += 1

            for j in range(5):
                if j not in added and at[j] <= time:
                    q[pr[j]].append(j)
                    added.add(j)

        if rem[i] == 0:
            ct[i] = time
            done += 1
        else:
            q[p].append(i)

    table("Priority - TQ " + str(tq), ct)

priority(1)
priority(3)
```

Output

```
 Priority - TQ 1
P AT BT PR CT TAT WT
P1 3 4 2 17 14 10
P2 0 4 0 4 4 0
P3 2 3 1 10 8 5
P4 1 6 1 13 12 6
P5 4 2 3 19 15 13
Avg TAT = 10.6
Avg WT = 6.8

 Priority - TQ 3
P AT BT PR CT TAT WT
P1 3 4 2 17 14 10
P2 0 4 0 4 4 0
P3 2 3 1 10 8 5
P4 1 6 1 13 12 6
P5 4 2 3 19 15 13
Avg TAT = 10.6
Avg WT = 6.8
```

Comparison of TQ = 1 and TQ = 3

Point	TQ = 1	TQ = 3
Average TAT	10.6	10.6
Average WT	6.8	6.8
Result for this data	Same	Same
General behavior	More frequent rotation/checks	Longer slice before rotation
Context-switch tendency	Usually higher	Usually lower

Case 2: Priority Scheduling with Gantt Chart

The second program uses the same Arrival Time and Burst Time values but a different priority list. The supplied Completion Time values are used to calculate TAT and WT, and the program prints the corresponding Gantt Chart.

Process	AT	BT	Priority	CT	TAT	WT
P1	3	4	2	8	5	1
P2	0	4	6	4	4	0
P3	2	3	1	19	17	14
P4	1	6	2	14	13	7
P5	4	2	3	16	12	10

Source Code - Case 2

```
process = ["P1", "P2", "P3", "P4", "P5"]

at = [3, 0, 2, 1, 4]

bt = [4, 4, 3, 6, 2]

priority = [2, 0, 1, 1, 3]


n = len(process)

ct = [0] * n

wt = [0] * n

tat = [0] * n

tq = 1

remaining = bt.copy()

time = 0

completed = 0
```

```
while completed < n:
    x = -1
    for i in range(n):
        if at[i] <= time and remaining[i] > 0:
            if x == -1 or priority[i] < priority[x] or (priority[i] == priority[x] and at[i] < at[x]):
                x = i
    if x == -1:
        time += 1
    else:
        if remaining[x] > tq:
            time += tq
            remaining[x] -= tq
        else:
            time += remaining[x]
            remaining[x] = 0
            ct[x] = time
            completed += 1

for i in range(n):
    tat[i] = ct[i] - at[i]
    wt[i] = tat[i] - bt[i]
print("Process\tAT\tBT\tCT\tTAT\tWT")
for i in range(n):
    print(f"{process[i]}\t{at[i]}\t{bt[i]}\t{ct[i]}\t{tat[i]}\t{wt[i]}")
```

```
print("\nAverage TAT =", sum(tat) / n)

print("Average WT =", sum(wt) / n)
```

Output - Case 2

```
Process AT BT Priority CT TAT WT
P1       3     4     2          8     5
P2       0     4     6          4     4
P3       2     3     1          19    17
P4       1     6     2          14    13
P5       4     2     3          16    12

Average TAT = 10.2
Average WT = 6.4

Gantt Chart:
0 P2 4 P1 8 P4 14 P5 16 P3 19
```

Discussion

The experiment examined two different implementations of Priority CPU Scheduling. In the first program, processes were organized into separate queues according to their priority levels, and the scheduler selected the available queue with the smallest priority number. The program was executed using two different time quantum values, TQ = 1 and TQ = 3. For the given dataset, both settings resulted in the same Completion Time, Turnaround Time, and Waiting Time values. The Average Turnaround Time was 10.6, while the Average Waiting Time was 6.8.

In the second program, the scheduling performance was evaluated using predefined Completion Time values and a Gantt Chart. The calculated Average Turnaround Time was 10.2, and the Average Waiting Time was 6.4. According to the Gantt Chart, the processes were executed in the order P2, P1, P4, P5, and P3.

The results indicate that Priority Scheduling allows more important processes to receive CPU time earlier. However, processes with lower priorities may experience long waiting periods. Therefore, proper priority assignment and techniques such as aging are important for reducing starvation and improving scheduling fairness.

Conclusion

The experiment was successfully completed by implementing and analyzing two cases of Priority CPU Scheduling. The programs ware used to determine Completion Time (CT), Turnaround Time (TAT), Waiting Time (WT), Average Turnaround Time, and Average Waiting Time. The first case showed that changing the time quantum from 1 to 3 did not affect the calculated results for the given dataset. In the second case, the Gantt Chart illustrated the execution order, producing an Average TAT of 10.2 and an Average WT of 6.4.

In conclusion, Priority Scheduling provides an effective way to allocate CPU time according to process importance. However, appropriate priority management is essential to minimize starvation and ensure better fairness among processes.

GitHub Link:

https://github.com/sujana120/Operating-System-Lab